

Problem 1 is more of the “easy” problem, Problem 2 is more of the “hard” one (because people on Ed were asking for a format like that). I’m not sure how hard Problem 3 will be (Brito)

Problem 1

- a) Sairoop and I were still discussing on the validity of some possible solutions at the time of writing, and I think Brito suggested students figure it out on their own, so I think I’ll leave this part for now.
- b) The idea is that to detect a negative cycle, we need to check if the *product* of consecutive exchange rates is greater than 1, but shortest paths only calculates *sums*. So what function can convert products to sum? The log function of course! students should convert all the edge weights to log-edge-weights, at this point the solution is pretty clear.
- c) Whatever algorithm they used for part (b), they should modify it to allow for returning a cycle, rather than just detecting it.

Tbh this question apparently is already pretty popular (whoops) so many students will probably cheat and find it online. But I think it is still good to set because it is not too difficult and allows students to explore using shortest-paths in different settings.

Problem 2

To solve this problem, there are two key issues to address: first is how to represent and efficiently measure shortest path between subsets rather than vertices. Second is how to deal with the negative weighted path.

- For the first issue, we add $2k$ vertices

$$\{s_1, s_2, \dots, s_k\} \cup \{t_1, t_2, \dots, t_k\}$$

and add weight-0 edges to connect s_i to every $v \in V_i$ and every $v \in V_i$ to t_i , where $i = 1, 2, \dots, k$.

- The second part is more general, I think there are a few ways to do it. The way I thought about it was like, “ok, I can’t do bellman ford repeatedly or floyd-warshall because that won’t fit in my time complexity, so what if I change the graph so that there are no negative weights, then I can use dijkstras repeatedly, which will fit in my time complexity”.

One idea is just to add constant factor to make everything positive. Unfortunately, this simple idea doesn’t work, intuitively because our two natural notions of “length” are incompatible—paths with more edges can have smaller total weight than paths with fewer edges. If we increase all edge weights at the same rate, paths with more edges get longer faster

than paths with fewer edges; as a result, the shortest path between two vertices might change. so our modification has to be a little more nuanced. This transformation can be given as follows:

- i. First, a new node q is added to the graph, connected by zero-weight edges to each of the other nodes.
- ii. Second, the Bellman–Ford algorithm is used, starting from the new vertex q , to find for each vertex v the minimum weight $h(v)$ of a path from q to v . If this step detects a negative cycle, the algorithm is terminated.
- iii. Next, the edges of the original graph are reweighted using the values computed by the Bellman–Ford algorithm: an edge from u to v , having length $w(u, v)$, is given the new length $w(u, v) + h(u) - h(v)$.

Think of it like the bank you’re leaving from makes you pay a “leaving” tax, but the bank you’re entering gives you a “entering” reward.

These new weights are non-negative *because* Bellman-Ford halted! This takes a bit of time to figure out, but it relies on the fact that bellman-ford tries to relax all edges of the form (s, t) such that $\text{dist}(q, s) + w_{(s,t)} < \text{dist}(q, t)$.

As for why we added the auxiliary vertex, If there is no suitable vertex that can reach everything, then no matter where we start Bellman-Ford, some of the resulting vertex prices will be infinite. Adding q doesn’t change the shortest paths between any pair of original vertices, because there are no paths into q .

To guide students on correctness, recommend them to check the proof of correctness for dijkstra’s, BF, and FW in the textbook; it will provide a good starting point.

If students are completely stuck, I would recommend first giving them the above advice (the two key issues to address), rather than the direct ways to address the issues, which is more for TAs to nudge if they’ve already started the problem but heading in a really wrong direction.

I think the implementation and correctness of this part, “if I transform the graph so that there are no negative weights”, will be the hardest for students to get (so feel free to give pointers on this part) but I think it is good for them to (at least attempt to) discover this on their own. if they use this transformation, it is super important their correctness is convincing, because it is easy to get it wrong.

The time complexity may seem a little strict, but this is actually a variant of a semi-popular algorithm given to first-course algorithm students, so I think second-course students should be able to get this bound.

Problem 3 matchings

States are indexed by T_v as defined in class.

We have two numbers: $\text{Matched}(v)$, $\text{NotMatched}(v)$, representing the size of the largest matching in T_v such that vertex v is matched, not matched, respectively.

The second coordinate recursively adds over all child, taking the best of the two choices, so

$$\text{NotMatched}(v) = \sum_{w:p(w)=v} \max(\text{Matched}(w), \text{NotMatched}(w))$$

For the first coordinate, let uv be the edge in the matching connected to v . Then we cannot match u in its subtree T_u and all other child are free. Hence

$$\text{Matched}(v) = 1 + \max_{u:p(u)=v} \left(\text{NotMatched}(u) + \sum_{w:w \neq u, p(w)=v} \max\{\text{Matched}(w), \text{NotMatched}(w)\} \right)$$

This is fine, but students may notice that the sum in the RHS of the last equation satisfies:

$$\sum_{w:w \neq u, p(w)=v} \max\{\text{Matched}(w), \text{NotMatched}(w)\} = \text{NotMatched}(v) - \max\{\text{Matched}(u), \text{NotMatched}(u)\}$$

following the equality for the second coordinate. This simplifies the calculations when populating the DP states.